

// Jivops

Guía completa — 30 lecciones

Arquitectura · State · Módulos · Seguridad · Colaboración y CI/CD ·
Operación a escala

Contenido

Módulo 1: Arquitectura de proyectos

- Lección 01 — Estructura de proyectos Terraform en producción
- Lección 02 — Providers y versionado
- Lección 03 — Variables y tfvars por entorno
- Lección 04 — Outputs y data sources
- Lección 05 — Terraform workspaces

Módulo 2: State management

- Lección 06 — Remote state — fundamentos
- Lección 07 — State locking con DynamoDB/backend
- Lección 08 — Import de infraestructura existente
- Lección 09 — terraform state mv y refactorización
- Lección 10 — Drift detection

Módulo 3: Módulos y reutilización

- Lección 11 — Módulos reutilizables — diseño
- Lección 12 — Registry de módulos (público y privado)
- Lección 13 — Composición de módulos
- Lección 14 — Versionado semántico de módulos
- Lección 15 — Testing de módulos con Terratest

Módulo 4: Seguridad

- Lección 16 — Gestión de secretos (Vault provider, SOPS)
- Lección 17 — IAM y principio de menor privilegio en Terraform
- Lección 18 — Políticas con Sentinel/OPA
- Lección 19 — Escaneo de seguridad (tfsec, Checkov)
- Lección 20 — Protección contra destroy accidental

Módulo 5: Colaboración y CI/CD

- Lección 21 — Terraform Cloud/Enterprise — introducción
- Lección 22 — CI/CD para Terraform (plan/apply automatizado)
- Lección 23 — Revisión de PRs con terraform plan output
- Lección 24 — Colaboración multi-equipo con workspaces
- Lección 25 — Rollback y recuperación ante errores de apply

Módulo 6: Operación a escala

Lección 26 — Terraform a gran escala — múltiples estados

Lección 27 — Optimización de performance (paralelismo, targeting)

Lección 28 — Gestión de dependencias entre módulos/stacks

Lección 29 — Documentación de infraestructura como código

Lección 30 — Proyecto final: infraestructura completa gestionada con Terraform en producción

Módulo 1: Arquitectura de proyectos

Lección 01 — Estructura de proyectos Terraform en producción

Separar archivos por propósito (main.tf, variables.tf, outputs.tf) facilita que el equipo mantenga el código. Separar entornos en directorios distintos evita aplicar cambios al entorno equivocado.

Lección 02 — Providers y versionado

Fijar la versión del provider evita que una actualización inesperada rompa la infraestructura sin previo aviso en el siguiente apply.

```
required_providers {  
  aws = { version = "~> 5.0" }  
}
```

Lección 03 — Variables y tfvars por entorno

Un archivo .tfvars por entorno permite reutilizar el mismo código Terraform para todos los entornos, cambiando solo los valores de entrada.

```
terraform apply -var-file=prod.tfvars
```

Lección 04 — Outputs y data sources

Un resource crea/gestiona infraestructura; un data source solo consulta información ya existente, útil para referenciar recursos no gestionados por este proyecto.

Lección 05 — Terraform workspaces

Los workspaces mantienen múltiples estados aislados con la misma configuración. Cuando los entornos difieren estructuralmente, conviene usar directorios separados en vez de workspaces.

```
terraform workspace new staging
```

Módulo 2: State management

Lección 06 — Remote state — fundamentos

Un backend remoto permite que el equipo colabore sin sobrescribirse el state entre sí, y lo protege de pérdida frente a mantenerlo solo en un laptop local.

```
terraform {  
  backend "s3"  
}
```

Lección 07 — State locking con DynamoDB/backend

El state locking previene que dos personas apliquen cambios simultáneamente y corrompan el archivo de estado con cambios sobrescribiéndose entre sí.

```
dynamodb_table = "terraform-locks"
```

Lección 08 — Import de infraestructura existente

terraform import trae bajo gestión un recurso creado manualmente. Es importante escribir el bloque resource correspondiente para que coincida, o Terraform podría intentar recrearlo.

```
terraform import aws_instance.mi_ec2 i-0123456789
```

Lección 09 — terraform state mv y refactorización

state mv mueve un recurso dentro del state sin destruirlo ni recrearlo en la infraestructura real, evitando downtime al reorganizar módulos.

Lección 10 — Drift detection

El drift es una diferencia entre el estado real de la infraestructura y lo que Terraform cree que existe, típicamente causado por cambios manuales fuera de Terraform.

Módulo 3: Módulos y reutilización

Lección 11 — Módulos reutilizables — diseño

Un módulo bien diseñado es configurable mediante variables de entrada, sin valores hardcoded, permitiendo adaptarlo a distintos proyectos o clientes.

Lección 12 — Registry de módulos (público y privado)

Publicar un módulo interno en un registry privado permite que otros equipos lo descubran y reutilicen con versionado formal, evitando reinventarlo.

Lección 13 — Composición de módulos

Componer módulos pequeños y especializados (VPC, EC2, RDS) en una configuración de nivel superior es más mantenible que un único módulo monolítico gigante.

```
module "app" {  
    vpc_id = module.vpc.id  
}
```

Lección 14 — Versionado semántico de módulos

Fijar un módulo a '~> 2.1' acepta actualizaciones menores compatibles pero no cambios mayores incompatibles, evitando romper producción sin previo aviso.

```
version = "~> 2.1"
```

Lección 15 — Testing de módulos con Terratest

Terratest permite escribir tests en Go que despliegan infraestructura real y validan su comportamiento, detectando problemas antes de que otros equipos dependan del módulo.

Módulo 4: Seguridad

Lección 16 — Gestión de secretos (Vault provider, SOPS)

Una contraseña en un .tfvars sin cifrar subido a Git queda expuesta en el historial del repositorio. El Vault provider permite leer secretos de forma centralizada sin exponerlos en el código.

Lección 17 — IAM y principio de menor privilegio en Terraform

Dar a Terraform solo los permisos estrictamente necesarios limita el alcance del daño posible si el código tiene un error o una credencial se filtra.

Lección 18 — Políticas con Sentinel/OPA

Una política Sentinel u OPA puede bloquear automáticamente un cambio que viole reglas de la organización antes de aplicarlo, como exigir cierta etiqueta obligatoria.

Lección 19 — Escaneo de seguridad (tfsec, Checkov)

tfsec analiza el código Terraform de forma estática, deteniendo configuraciones inseguras (como un bucket S3 público) antes de que se apliquen realmente.

Lección 20 — Protección contra destroy accidental

'prevent_destroy = true' bloquea que un recurso crítico (como una base de datos de producción) se destruya accidentalmente con un apply o destroy posterior.

```
lifecycle {
  prevent_destroy = true
}
```

Módulo 5: Colaboración y CI/CD

Lección 21 — Terraform Cloud/Enterprise — introducción

Terraform Cloud ejecuta planes y applies de forma remota y centralizada, permitiendo restringir quién puede aplicar cambios y exigir aprobación antes de un apply en producción.

Lección 22 — CI/CD para Terraform (plan/apply automatizado)

Un pipeline típico corre validate, plan, revisión humana y apply tras aprobación — esa revisión permite confirmar exactamente qué cambios ocurrirán antes de aplicarlos de verdad.

```
terraform validate
```

Lección 23 — Revisión de PRs con terraform plan output

Publicar el output del plan como comentario en el PR permite al equipo revisar el cambio real y exacto, sin depender solo de la descripción escrita.

Lección 24 — Colaboración multi-equipo con workspaces

Separar el state por dominio (red, aplicación, datos) reduce el acoplamiento entre equipos, evitando que un cambio de uno bloquee accidentalmente a otro.

Lección 25 — Rollback y recuperación ante errores de apply

Terraform no tiene un comando de rollback automático; la forma real de recuperación es revertir el código en Git a una versión anterior conocida que funcionaba.

```
git revert commit_id
```


Módulo 6: Operación a escala

Lección 26 — Terraform a gran escala — múltiples estados

Dividir una infraestructura muy grande en múltiples states pequeños reduce el tiempo de plan/apply y limita el impacto de un fallo a esa parte específica.

Lección 27 — Optimización de performance (paralelismo, targeting)

-parallelism controla cuántas operaciones se ejecutan simultáneamente. -target acelera un cambio puntual urgente, pero usarlo con frecuencia puede dejar el state inconsistente.

```
terraform apply -parallelism=20
```

Lección 28 — Gestión de dependencias entre módulos/stacks

terraform_remote_state lee el output de otro state de forma automática y actualizada, más confiable que copiar manualmente un valor entre proyectos.

Lección 29 — Documentación de infraestructura como código

terraform-docs genera documentación en Markdown directamente del código, evitando que quede desactualizada como pasaría con documentación mantenida a mano.

```
terraform-docs markdown .
```

Lección 30 — Proyecto final: infraestructura completa gestionada con Terraform en producción

Se integra todo lo anterior: módulos reutilizables, remote state con locking, escaneo de seguridad, CI/CD con revisión de plan, políticas y documentación — el nivel esperado de una infraestructura lista para producción real.